

IPSignal Fraud Analysis - April 2026

Client-Facing Updated Analysis, Code, Visualizations and Rule Recommendation Framework

Scope update: This document updates the prior analysis based on presentation feedback. The client-facing version narrows the analysis window to April 2026, removes IPv6 references, includes Score 3, recommends velocity and layered rules by integration point, and adds triggered transaction counts and deduplicated cumulative rule impact.

Table of Contents

- 1. Executive objective
- 2. Feedback-driven refinements
- 3. Analytical principles
- 4. Data preparation and April 2026 universe
- 5. Baseline metrics
- 6. IPSS distribution including Score 3
- 7. Fraud capture for Scores 3 and 4
- 8. Reason code taxonomy and supplemental mapping
- 9. Reason family performance
- 10. Residential proxy and RP-tier stratification
- 11. Velocity metrics by integration point
- 12. Automated thresholds and velocity rule backtesting
- 13. Candidate layered rules including Score 3
- 14. Triggered transactions and rule impact
- 15. Deduplicated cumulative rule impact
- 16. Visualizations for the updated deck
- 17. Recommendation extension: transaction volume by rule
- 18. Recommendation extension: deduplication and portfolio impact
- 19. Final client-facing recommendation table
- 20. Suggested client storyline
- 21. Full Python/DuckDB code block

1. Executive objective

- Evaluate IPSignal performance using fraud_direct as the confirmed fraud label.
- Use April 2026 as the primary analysis window to allow fraud evidence to mature while avoiding the June 30 evidence spike.
- Move from score-only analysis to layered decisioning: Score + Reason Family + Velocity + Integration Point.
- Include Score 3 in the analysis because feedback indicated its fraud rate may be comparable to Score 4 in some segments.
- Produce exact rule names, triggered transaction counts, fraud capture, fraud lift, and friction metrics by integration point.

2. Feedback-driven refinements

Feedback Area	Updated Requirement	Reason
Analysis window	Use April 1-30, 2026 only	Reduces evidence maturity bias and avoids June 30 spike/noise.
IPSS distribution	Include Score 3	Score 3 may show fraud rate comparable to Score 4 in some segments.
IPv4/IPv6	Remove from client-facing deck	Comparison is valuable internally but not actionable for client-facing rules at this stage.
Velocity	Present by integration point	Rules cannot be segmented by IP version, so velocity should be operationalized by flow.
Fraud capture	Emphasize Scores 3 and 4	Shows value beyond Score 5 and supports

		layered rule design.
Reason families	Include supplemental mapping file	Provides transparency for reason family groupings and categorization.
Rules	Show exact rule names per integration point	Makes recommendations implementable and easier to review.
Recommendations	Remove references to IPv6	Keep the client-facing narrative focused on actionable score/reason/velocity decisions.

3. Analytical principles

- Do not recommend blanket deny rules using Score 4 or Score 5 alone. Scores 4-5 can represent a large portion of traffic and may create overblocking risk.
- Evaluate Score 3, 4, and 5 separately: Score 5 may capture more fraud by volume, while Score 3 or 4 may show stronger fraud rate in specific segments.
- Prioritize reason family signals, especially Anonymous / Privacy and Attack / Bot / Spam, because they improve precision.
- Prioritize velocity rules by integration point, not by IP version.
- Measure each rule by triggered transactions, triggered transaction share, triggered frauds, precision fraud rate, fraud lift, fraud capture, friction, unique devices, and unique accounts.
- Measure rules both individually and cumulatively using deduplicated transaction impact.

4. Data preparation and April 2026 universe

Implementation note: The April universe should include scored transactions only, using fraud_direct as the label. The code below assumes a base_all table or view has already been created from the parquet files. If the raw table name is different, replace base_all accordingly.

Code - setup and April universe

```
import duckdb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

con = duckdb.connect("ipsignal_fraud_analysis.duckdb")
con.execute("PRAGMA threads=8;")
con.execute("PRAGMA memory_limit='8GB';")
con.execute("PRAGMA temp_directory='/tmp/duckdb_temp';")

OUTPUT_DIR = Path("client_facing_april2026_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

ANALYSIS_START = "2026-04-01"
ANALYSIS_END = "2026-05-01"

def save_plot(filename):
    path = OUTPUT_DIR / filename
    plt.tight_layout()
    plt.savefig(path, dpi=300, bbox_inches="tight")
    plt.show()
    print(f"Saved: {path}")

con.execute(f"""
CREATE OR REPLACE TABLE april_base AS
SELECT
    UUID,
    TRY_CAST(MSG_TIMESTAMP AS TIMESTAMP) AS ts,
    TRY_CAST(MSG_DAY AS DATE) AS msg_day,
    SUBSCRIBER_ID,
    DEVICE_ID,
    DEVICE_TYPE,
    DEVICE_NEW,
    ACCOUNT_CODE,
    ACCOUNT_NEW,
    LOWER(TRIM(RESULT)) AS result,
    COALESCE(TRY_CAST(fraud_direct AS INTEGER), 0) AS fraud_direct,
    COALESCE(TRY_CAST(fraud_indirect AS INTEGER), 0) AS fraud_indirect,
    TRY_CAST(IPSIGNAL_SCORE_1 AS INTEGER) AS ipsignal_score,
    TRY_CAST(IPSIGNAL_LAST_SCORE_1 AS INTEGER) AS ipsignal_last_score,
    RESULT_CODES_1 AS result_codes,
    CODE_1 AS code_1,
    CODE_2 AS code_2,
```

```

CODE_3 AS code_3,
CODE_4 AS code_4,
CODE_5 AS code_5,
CODE_6 AS code_6,
INTEGRATION_POINT_NAME AS integration_point,
REALIP_ADDRESS_1 AS real_ipaddress,
REALIPLC_CITY AS ip_city,
REALIPLC_COUNTRY AS ip_country,
REALIPLC_REGION AS ip_region
FROM base_all
WHERE TRY_CAST(MSG_TIMESTAMP AS TIMESTAMP) >= TIMESTAMP '{ANALYSIS_START}'
AND TRY_CAST(MSG_TIMESTAMP AS TIMESTAMP) < TIMESTAMP '{ANALYSIS_END}'
AND TRY_CAST(IPSIGNAL_SCORE_1 AS INTEGER) BETWEEN 1 AND 5;
""")

```

5. Baseline metrics

- The April baseline is the comparison point for fraud lift.
- The baseline should be calculated for the same universe used in the client-facing score and rule analysis.
- All rule metrics should be compared to this baseline, not to the full Jan-June population.

Code - April baseline

```

baseline = con.execute("""
SELECT
    COUNT(*) AS txns,
    SUM(fraud_direct) AS frauds,
    AVG(fraud_direct) AS fraud_rate,
    SUM(CASE WHEN result = 'allow' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS allow_rate,
    SUM(CASE WHEN result = 'review' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS review_rate,
    SUM(CASE WHEN result = 'deny' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS deny_rate
FROM april_base;
""").df()

overall_fraud_rate = baseline.loc[0, "fraud_rate"]
baseline.to_csv(OUTPUT_DIR / "baseline_april2026.csv", index=False)
baseline

```

6. IPSS distribution including Score 3

- Keep Scores 1-5 visible, but explicitly call out Score 3.
- Do not say that Score 4 and Score 5 alone are sufficient for decisioning.
- Use traffic share and fraud rate together: traffic share shows volume exposure; fraud rate shows risk intensity.

Code - score summary

```

score_summary = con.execute("""
SELECT
    ipsignal_score,
    CASE
        WHEN ipsignal_score = 5 THEN 'Very High'
        WHEN ipsignal_score = 4 THEN 'High'
        WHEN ipsignal_score = 3 THEN 'Medium-High'
        WHEN ipsignal_score = 2 THEN 'Low'
        WHEN ipsignal_score = 1 THEN 'Very Low'
    END AS risk_level,
    COUNT(*) AS txns,
    SUM(fraud_direct) AS frauds,
    AVG(fraud_direct) AS fraud_rate,
    SUM(CASE WHEN result = 'allow' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS allow_rate,
    SUM(CASE WHEN result = 'review' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS review_rate,
    SUM(CASE WHEN result = 'deny' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS deny_rate
FROM april_base
GROUP BY 1, 2
ORDER BY ipsignal_score;
""").df()

score_summary["traffic_share"] = score_summary["txns"] / score_summary["txns"].sum()
score_summary["fraud_capture"] = score_summary["frauds"] / score_summary["frauds"].sum()
score_summary["fraud_lift"] = score_summary["fraud_rate"] / overall_fraud_rate
score_summary.to_csv(OUTPUT_DIR / "score_summary_april2026.csv", index=False)
score_summary

```

Visualization - score distribution and fraud rate

```

x = np.arange(len(score_summary))
width = 0.35

plt.figure(figsize=(10, 6))
plt.bar(x - width / 2, score_summary["traffic_share"] * 100, width, label="Traffic Share (%)")
plt.bar(x + width / 2, score_summary["fraud_rate"] * 100, width, label="Fraud Rate (%)")
plt.title("IPSignal Score Distribution and Fraud Rate - April 2026")
plt.xlabel("IPSignal Score")
plt.ylabel("Percentage")
plt.xticks(x, [f"Score {s}" for s in score_summary["ipsignal_score"]])
plt.legend()
save_plot("01_ipss_distribution_score_1_to_5_april.png")

```

7. Fraud capture for Scores 3 and 4

- This directly addresses the feedback to emphasize fraud capture achieved by Scores 3 and 4.
- The goal is to show whether Scores 3 and 4 capture meaningful fraud and whether they should be included in layered rule design.
- Score 3 should be assessed with secondary signals, not used as a standalone decision rule.

Code - Score 3/4 focus

```

score_capture_focus = score_summary[score_summary["ipsignal_score"].isin([3, 4, 5]).copy()

plt.figure(figsize=(8, 5))
plt.bar(score_capture_focus["ipsignal_score"].astype(str), score_capture_focus["fraud_capture"] * 100)
plt.title("Fraud Capture by IPSignal Score - Focus on Scores 3 and 4")
plt.xlabel("IPSignal Score")
plt.ylabel("Fraud Capture (%)")
save_plot("02_fraud_capture_scores_3_4_5_april.png")

score_3_4_comparison = score_summary[score_summary["ipsignal_score"].isin([3, 4])][["ipsignal_score", "risk_level", "txns", "frauds", "fraud_rate", "traffic_share", "fraud_capture", "fraud_lift"]]
score_3_4_comparison.to_csv(OUTPUT_DIR / "score_3_4_comparison.csv", index=False)
score_3_4_comparison

```

8. Reason code taxonomy and supplemental mapping

- Reason codes should be converted from wide columns code_1 to code_6 into a long table.
- A supplemental mapping file should be created to show how each reason code maps to its reason family.
- The supplemental file requested by reviewers is generated as supplemental_reason_family_groupings.csv.

Code - reason code long table and supplemental mapping

```

con.execute("""
CREATE OR REPLACE TABLE april_reason_long AS
SELECT
    uuid,
    ts,
    integration_point,
    device_id,
    account_code,
    result,
    fraud_direct,
    ipsignal_score,
    reason_code_raw,
    REGEXP_REPLACE(
        UPPER(TRIM(reason_code_raw)),
        '[_\\s\\-]+',
        '-',
        'g'
    ) AS reason_code_norm
FROM april_base
CROSS JOIN UNNEST([code_1, code_2, code_3, code_4, code_5, code_6]) AS t(reason_code_raw)
WHERE reason_code_raw IS NOT NULL
    AND UPPER(TRIM(reason_code_raw)) NOT IN ('NONE', 'NULL', '');
""")

con.execute("""
CREATE OR REPLACE TABLE reason_code_dim AS
SELECT *

```

```

FROM (
  VALUES
    ('ANAC', 'Anonymous / Privacy'),
    ('ANPR', 'Anonymous / Privacy'),
    ('ANOU', 'Anonymous / Privacy'),
    ('TR', 'Anonymous / Privacy'),
    ('ATK1', 'Attack / Bot / Spam'),
    ('ATK2', 'Attack / Bot / Spam'),
    ('BOT_01', 'Attack / Bot / Spam'),
    ('SPM1', 'Attack / Bot / Spam'),
    ('SPM2', 'Attack / Bot / Spam'),
    ('BC', 'Hosting / Infrastructure'),
    ('BGP', 'Hosting / Infrastructure'),
    ('MGW', 'Hosting / Infrastructure'),
    ('SRVR_01', 'Hosting / Infrastructure'),
    ('SRVR_02', 'Hosting / Infrastructure'),
    ('SRVR_03', 'Hosting / Infrastructure'),
    ('CORP_IP', 'Institutional / Managed Network'),
    ('EDU', 'Institutional / Managed Network'),
    ('GOVT', 'Institutional / Managed Network'),
    ('LIB', 'Institutional / Managed Network'),
    ('CL', 'Home / Consumer Network'),
    ('HF', 'Home / Consumer Network'),
    ('HOM', 'Home / Consumer Network'),
    ('INPR_IP', 'Residential Proxy'),
    ('RGPR_IP', 'Residential Proxy'),
    ('RP1', 'Residential Proxy'),
    ('RP2', 'Residential Proxy'),
    ('RP3', 'Residential Proxy'),
    ('USGL', 'Geo / Location Signal'),
    ('USGH', 'Geo / Location Signal'),
    ('ML1', 'Behavioral Intelligence'),
    ('ML2', 'Behavioral Intelligence'),
    ('WL_RSK', 'Watchlist / Rule List'),
    ('WL_RL', 'Watchlist / Rule List')
) AS t(reason_code_norm, reason_family);
"""

reason_family_mapping = con.execute("""
SELECT *
FROM reason_code_dim
ORDER BY reason_family, reason_code_norm;
""").df()
reason_family_mapping.to_csv(OUTPUT_DIR / "supplemental_reason_family_groupings.csv", index=False)
reason_family_mapping

```

9. Reason family performance

- This is one of the strongest precision layers and should remain prominent in the client-facing deck.
- Rank families by fraud lift and fraud rate, not only by transaction volume.
- Anonymous / Privacy and Attack / Bot / Spam should be evaluated as priority signals for Review, Step-up, or Deny candidates.

Code - reason family performance

```

reason_family_summary = con.execute("""
WITH enriched AS (
  SELECT
    r.*,
    COALESCE(d.reason_family, 'Unmapped / Needs Review') AS reason_family
  FROM april_reason_long r
  LEFT JOIN reason_code_dim d
    ON r.reason_code_norm = d.reason_code_norm
)
SELECT
  reason_family,
  COUNT(DISTINCT uuid) AS txns,
  SUM(fraud_direct) AS frauds,
  AVG(fraud_direct) AS fraud_rate,
  COUNT(DISTINCT uuid) * 1.0 / (SELECT COUNT(*) FROM april_base) AS traffic_share,
  SUM(fraud_direct) * 1.0 / NULLIF((SELECT SUM(fraud_direct) FROM april_base), 0) AS fraud_capture
FROM enriched
GROUP BY 1
HAVING COUNT(DISTINCT uuid) >= 30
ORDER BY fraud_rate DESC;
""").df()
reason_family_summary["fraud_lift"] = reason_family_summary["fraud_rate"] / overall_fraud_rate
reason_family_summary.to_csv(OUTPUT_DIR / "reason_family_performance_april.csv", index=False)
reason_family_summary

```

Visualization - reason family fraud lift

```

plot_df = reason_family_summary.sort_values("fraud_lift", ascending=True)

plt.figure(figsize=(10, 6))
plt.barh(plot_df["reason_family"], plot_df["fraud_lift"])
plt.axvline(1, linestyle="--")
plt.title("Reason Family Fraud Lift - April 2026")
plt.xlabel("Fraud Lift vs Baseline")
plt.ylabel("Reason Family")
save_plot("03_reason_family_fraud_lift_april.png")

```

10. Residential proxy and RP-tier stratification

- Keep residential proxy analysis as a specific drilldown within reason family analysis.
- Evaluate RP1, RP2, and RP3 independently; do not assume the ordinal tier always matches observed fraud rate.
- Use RP tier in layered rules, especially when combined with Score 3 or 4.

Code - RP-tier stratification

```

con.execute("""
CREATE OR REPLACE TABLE april_reason_flags AS
WITH enriched AS (
    SELECT
        r.*,
        COALESCE(d.reason_family, 'Unmapped / Needs Review') AS reason_family
    FROM april_reason_long r
    LEFT JOIN reason_code_dim d
        ON r.reason_code_norm = d.reason_code_norm
)
SELECT
    uuid,
    MAX(CASE WHEN reason_family = 'Anonymous / Privacy' THEN 1 ELSE 0 END) AS has_anonymous_privacy,
    MAX(CASE WHEN reason_family = 'Attack / Bot / Spam' THEN 1 ELSE 0 END) AS has_attack_bot_spam,
    MAX(CASE WHEN reason_family = 'Residential Proxy' THEN 1 ELSE 0 END) AS has_residential_proxy,
    MAX(CASE WHEN reason_family = 'Hosting / Infrastructure' THEN 1 ELSE 0 END) AS has_hosting_infra,
    MAX(CASE WHEN reason_family = 'Behavioral Intelligence' THEN 1 ELSE 0 END) AS has_behavioral_intel,
    MAX(CASE WHEN reason_family = 'Watchlist / Rule List' THEN 1 ELSE 0 END) AS has_watchlist,
    MAX(CASE WHEN reason_code_norm = 'RP1' THEN 1 ELSE 0 END) AS has_rp1,
    MAX(CASE WHEN reason_code_norm = 'RP2' THEN 1 ELSE 0 END) AS has_rp2,
    MAX(CASE WHEN reason_code_norm = 'RP3' THEN 1 ELSE 0 END) AS has_rp3,
    CASE
        WHEN MAX(CASE WHEN reason_code_norm = 'RP3' THEN 1 ELSE 0 END) = 1 THEN 3
        WHEN MAX(CASE WHEN reason_code_norm = 'RP2' THEN 1 ELSE 0 END) = 1 THEN 2
        WHEN MAX(CASE WHEN reason_code_norm = 'RP1' THEN 1 ELSE 0 END) = 1 THEN 1
        ELSE 0
    END AS max_rp_tier,
    COUNT(DISTINCT reason_code_norm) AS number_reason_codes
FROM enriched
GROUP BY uuid;
""")

con.execute("""
CREATE OR REPLACE TABLE april_enriched AS
SELECT
    b.*,
    COALESCE(f.has_anonymous_privacy, 0) AS has_anonymous_privacy,
    COALESCE(f.has_attack_bot_spam, 0) AS has_attack_bot_spam,
    COALESCE(f.has_residential_proxy, 0) AS has_residential_proxy,
    COALESCE(f.has_hosting_infra, 0) AS has_hosting_infra,
    COALESCE(f.has_behavioral_intel, 0) AS has_behavioral_intel,
    COALESCE(f.has_watchlist, 0) AS has_watchlist,
    COALESCE(f.has_rp1, 0) AS has_rp1,
    COALESCE(f.has_rp2, 0) AS has_rp2,
    COALESCE(f.has_rp3, 0) AS has_rp3,
    COALESCE(f.max_rp_tier, 0) AS max_rp_tier,
    COALESCE(f.number_reason_codes, 0) AS number_reason_codes
FROM april_base b
LEFT JOIN april_reason_flags f
    ON b.uuid = f.uuid;
""")

rp_summary = con.execute("""
SELECT
    integration_point,
    max_rp_tier,
    COUNT(*) AS txns,
    SUM(fraud_direct) AS frauds,
    AVG(fraud_direct) AS fraud_rate,
    COUNT(*) * 1.0 / SUM(COUNT(*)) OVER (PARTITION BY integration_point) AS traffic_share,
    SUM(fraud_direct) * 1.0 / NULLIF(SUM(SUM(fraud_direct)) OVER (PARTITION BY integration_point), 0) AS fraud_capture,
    AVG(ipsignal_score) AS avg_score

```

```

FROM april_enriched
WHERE max_rp_tier > 0
GROUP BY 1, 2
ORDER BY integration_point, max_rp_tier;
""").df()
rp_summary["fraud_lift"] = rp_summary["fraud_rate"] / overall_fraud_rate
rp_summary.to_csv(OUTPUT_DIR / "residential_proxy_rp_tier_summary_april.csv", index=False)
rp_summary

```

Visualization - RP-tier lift by integration point

```

for integration in rp_summary["integration_point"].dropna().unique():
    temp = rp_summary[rp_summary["integration_point"] == integration].copy()
    temp = temp.sort_values("max_rp_tier")
    plt.figure(figsize=(8, 5))
    plt.bar(temp["max_rp_tier"].astype(str), temp["fraud_lift"])
    plt.axhline(1, linestyle="--")
    plt.title(f"Residential Proxy Fraud Lift by RP Tier - {integration}")
    plt.xlabel("Max RP Tier")
    plt.ylabel("Fraud Lift vs Baseline")
    save_plot(f"04_rp_tier_fraud_lift_{integration}.png")

```

11. Velocity metrics by integration point

- Velocity rules should be presented by integration point, as requested in the feedback.
- Recommended windows: 5 minutes, 15 minutes, 1 hour, and 24 hours.
- Recommended entities: device_id, account_code, and real_ipaddress.
- The code below partitions velocities by integration_point to make thresholds operational by flow.

Code - velocity table by integration point

```

con.execute("""
CREATE OR REPLACE TABLE april_velocity AS
SELECT
    *,
    COUNT(*) OVER (
        PARTITION BY integration_point, device_id
        ORDER BY ts
        RANGE BETWEEN INTERVAL 5 MINUTE PRECEDING AND CURRENT ROW
    ) AS device_txn_5m,
    COUNT(*) OVER (
        PARTITION BY integration_point, device_id
        ORDER BY ts
        RANGE BETWEEN INTERVAL 15 MINUTE PRECEDING AND CURRENT ROW
    ) AS device_txn_15m,
    COUNT(*) OVER (
        PARTITION BY integration_point, device_id
        ORDER BY ts
        RANGE BETWEEN INTERVAL 1 HOUR PRECEDING AND CURRENT ROW
    ) AS device_txn_1h,
    COUNT(*) OVER (
        PARTITION BY integration_point, device_id
        ORDER BY ts
        RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW
    ) AS device_txn_24h,
    COUNT(*) OVER (
        PARTITION BY integration_point, account_code
        ORDER BY ts
        RANGE BETWEEN INTERVAL 5 MINUTE PRECEDING AND CURRENT ROW
    ) AS account_txn_5m,
    COUNT(*) OVER (
        PARTITION BY integration_point, real_ipaddress
        ORDER BY ts
        RANGE BETWEEN INTERVAL 5 MINUTE PRECEDING AND CURRENT ROW
    ) AS ip_txn_5m
FROM april_enriched;
""")

```

12. Automated thresholds and velocity rule backtesting

- Compute p90, p95, and p99 thresholds per velocity feature and integration point.
- Backtest each velocity threshold independently.
- Select the strongest rules per integration point using fraud lift, fraud capture, and friction.

Code - velocity thresholds

```

velocity_features = [
    "device_txn_5m",
    "device_txn_15m",
    "device_txn_1h",
    "device_txn_24h",
    "account_txn_5m",
    "ip_txn_5m"
]

threshold_queries = []
for feature in velocity_features:
    threshold_queries.append(f"""
SELECT
    integration_point,
    '{feature}' AS velocity_feature,
    QUANTILE_CONT({feature}, 0.90) AS p90_threshold,
    QUANTILE_CONT({feature}, 0.95) AS p95_threshold,
    QUANTILE_CONT({feature}, 0.99) AS p99_threshold
FROM april_velocity
GROUP BY 1
""")

velocity_thresholds = con.execute(" UNION ALL ".join(threshold_queries)).df()
velocity_thresholds.to_csv(OUTPUT_DIR / "velocity_thresholds_by_integration_point.csv", index=False)
velocity_thresholds

```

Code - velocity rule backtest and top velocity rules

```

velocity_rule_results = []

for _, row in velocity_thresholds.iterrows():
    integration = str(row["integration_point"]).replace("'", "")
    feature = row["velocity_feature"]

    for threshold_name in ["p90_threshold", "p95_threshold", "p99_threshold"]:
        threshold_value = row[threshold_name]
        if pd.isna(threshold_value):
            continue

        query = f"""
WITH universe AS (
    SELECT *
    FROM april_velocity
    WHERE integration_point = '{integration}'
),
triggered AS (
    SELECT *
    FROM universe
    WHERE {feature} >= {threshold_value}
)
SELECT
    '{integration}' AS integration_point,
    'Velocity | {integration} | {feature} >= {threshold_name}' AS rule_name,
    '{feature}' AS velocity_feature,
    '{threshold_name}' AS threshold_type,
    {threshold_value} AS threshold_value,
    COUNT(*) AS triggered_txns,
    COUNT(DISTINCT uuid) AS triggered_unique_txns,
    COUNT(DISTINCT device_id) AS unique_devices,
    COUNT(DISTINCT account_code) AS unique_accounts,
    SUM(fraud_direct) AS triggered_frauds,
    AVG(fraud_direct) AS precision_fraud_rate,
    COUNT(*) * 1.0 / NULLIF((SELECT COUNT(*) FROM universe), 0) AS triggered_txn_share,
    COUNT(*) * 1.0 / NULLIF((SELECT COUNT(*) FROM universe), 0) AS friction_rate,
    SUM(fraud_direct) * 1.0 / NULLIF((SELECT SUM(fraud_direct) FROM universe), 0) AS fraud_capture_rate,
    AVG(fraud_direct) / NULLIF((SELECT AVG(fraud_direct) FROM universe), 0) AS fraud_lift
FROM triggered;
"""
        velocity_rule_results.append(con.execute(query).df())

velocity_rule_results = pd.concat(velocity_rule_results, ignore_index=True)
velocity_rule_results = velocity_rule_results.sort_values(
    ["integration_point", "fraud_lift", "fraud_capture_rate"],
    ascending=[True, False, False]
)
velocity_rule_results.to_csv(OUTPUT_DIR / "velocity_rule_backtest_by_integration_point.csv", index=False)

top_velocity_rules = (
    velocity_rule_results
    .query("triggered_txns > 0")
    .sort_values(["integration_point", "fraud_lift", "fraud_capture_rate"], ascending=[True, False, False])
    .groupby("integration_point")
    .head(5)
)

```



```

        .reset_index(drop=True)
    )
    top_velocity_rules.to_csv(OUTPUT_DIR / "recommended_velocity_rules_by_integration_point.csv", index=False)
    top_velocity_rules

```

Visualization - top velocity rules by integration point

```

for integration in top_velocity_rules["integration_point"].unique():
    temp = top_velocity_rules[top_velocity_rules["integration_point"] == integration]
    temp = temp.sort_values("fraud_lift", ascending=True)

    plt.figure(figsize=(10, 6))
    plt.barh(temp["rule_name"], temp["fraud_lift"])
    plt.axvline(1, linestyle="--")
    plt.title(f"Recommended Velocity Rules - {integration}")
    plt.xlabel("Fraud Lift")
    plt.ylabel("Rule Name")
    save_plot(f"05_recommended_velocity_rules_{integration}.png")

```

13. Candidate layered rules including Score 3

- Candidate rules should include Score 3 where paired with high-risk reason families or high velocity.
- Rules should be reported separately by integration point.
- Rule names should be explicit and ready for discussion with rule owners.

Code - rule hit table with exact rule names

```

con.execute("""
CREATE OR REPLACE TABLE april_rule_hits AS
WITH universe AS (
    SELECT *
    FROM april_velocity
)
SELECT integration_point, uuid, 'Score 3/4 + Anonymous Privacy' AS rule_name, 1 AS rule_priority, 'Review / step-up candidate' AS recommended_action
FROM universe
WHERE ipsignal_score IN (3, 4) AND has_anonymous_privacy = 1
UNION ALL
SELECT integration_point, uuid, 'Score 3/4 + Attack Bot Spam' AS rule_name, 2 AS rule_priority, 'Review / step-up candidate' AS recommended_action
FROM universe
WHERE ipsignal_score IN (3, 4) AND has_attack_bot_spam = 1
UNION ALL
SELECT integration_point, uuid, 'Score 3 + Anonymous Privacy' AS rule_name, 3 AS rule_priority, 'Review / step-up candidate' AS recommended_action
FROM universe
WHERE ipsignal_score = 3 AND has_anonymous_privacy = 1
UNION ALL
SELECT integration_point, uuid, 'Score 4 + Anonymous Privacy' AS rule_name, 4 AS rule_priority, 'Review / step-up candidate' AS recommended_action
FROM universe
WHERE ipsignal_score = 4 AND has_anonymous_privacy = 1
UNION ALL
SELECT integration_point, uuid, 'Score 4 + RP3' AS rule_name, 5 AS rule_priority, 'Deny candidate if friction tolerance allows' AS recommended_action
FROM universe
WHERE ipsignal_score = 4 AND has_rp3 = 1
UNION ALL
SELECT integration_point, uuid, 'Score 3/4 + Residential Proxy' AS rule_name, 6 AS rule_priority, 'Monitor / secondary signal' AS recommended_action
FROM universe
WHERE ipsignal_score IN (3, 4) AND has_residential_proxy = 1
UNION ALL
SELECT integration_point, uuid, 'Score 3/4 + High Device Velocity 24h' AS rule_name, 7 AS rule_priority, 'Review / step-up candidate' AS recommended_action
FROM universe
WHERE ipsignal_score IN (3, 4) AND device_txn_24h >= 9
UNION ALL
SELECT integration_point, uuid, 'Score 3/4 + High Device Velocity 1h' AS rule_name, 8 AS rule_priority, 'Review / step-up candidate' AS recommended_action
FROM universe
WHERE ipsignal_score IN (3, 4) AND device_txn_1h >= 3
UNION ALL
SELECT integration_point, uuid, 'Score 3/4 + Anonymous Privacy + High Velocity 24h' AS rule_name, 9 AS rule_priority, 'Deny candidate if friction tolerance allows' AS recommended_action
FROM universe
WHERE ipsignal_score IN (3, 4) AND has_anonymous_privacy = 1 AND device_txn_24h >= 9
UNION ALL
SELECT integration_point, uuid, 'Score 3/4 + Attack Bot Spam + High Velocity 24h' AS rule_name, 10 AS rule_priority, 'Deny candidate if friction tolerance allows' AS recommended_action
FROM universe

```

```
WHERE ipsignal_score IN (3, 4) AND has_attack_bot_spam = 1 AND device_txn_24h >= 9;
""")
```

14. Triggered transactions and rule impact

- This section implements the new recommendation: add transaction counts generated by each rule.
- Triggered transactions should be shown alongside triggered transaction share, fraud capture, precision, lift, and friction.
- Unique devices and unique accounts help quantify customer concentration and operational impact.

Code - individual rule backtest with triggered transactions

```
rule_backtest = con.execute("""
WITH universe AS (
    SELECT *
    FROM april_velocity
),
rule_txns AS (
    SELECT
        h.integration_point,
        h.rule_name,
        MIN(h.rule_priority) AS rule_priority,
        MAX(h.recommended_action) AS recommended_action,
        COUNT(DISTINCT h.uuid) AS triggered_txns,
        SUM(u.fraud_direct) AS triggered_frauds,
        AVG(u.fraud_direct) AS precision_fraud_rate,
        COUNT(DISTINCT u.device_id) AS unique_devices,
        COUNT(DISTINCT u.account_code) AS unique_accounts,
        SUM(CASE WHEN u.fraud_direct = 0 THEN 1 ELSE 0 END) AS legitimate_txns_touched
    FROM april_rule_hits h
    JOIN universe u
        ON h.uuid = u.uuid
        AND h.integration_point = u.integration_point
    GROUP BY 1, 2
),
universe_summary AS (
    SELECT
        integration_point,
        COUNT(DISTINCT uuid) AS total_txns,
        SUM(fraud_direct) AS total_frauds,
        AVG(fraud_direct) AS baseline_fraud_rate
    FROM universe
    GROUP BY 1
)
SELECT
    r.integration_point,
    r.rule_name,
    r.rule_priority,
    r.recommended_action,
    r.triggered_txns,
    r.triggered_txns * 1.0 / NULLIF(u.total_txns, 0) AS triggered_txn_share,
    r.triggered_frauds,
    r.precision_fraud_rate,
    r.precision_fraud_rate / NULLIF(u.baseline_fraud_rate, 0) AS fraud_lift,
    r.triggered_frauds * 1.0 / NULLIF(u.total_frauds, 0) AS fraud_capture_rate,
    r.legitimate_txns_touched * 1.0 / NULLIF(u.total_txns, 0) AS friction_rate,
    r.unique_devices,
    r.unique_accounts,
    r.legitimate_txns_touched
FROM rule_txns r
JOIN universe_summary u
    ON r.integration_point = u.integration_point
ORDER BY r.integration_point, fraud_lift DESC, fraud_capture_rate DESC;
""").df()

rule_backtest.to_csv(OUTPUT_DIR / "candidate_rule_backtest_by_integration_point.csv", index=False)
rule_backtest.head(50)
```

Code - top candidate rules per integration point

```
top_candidate_rules = (
    rule_backtest
    .query("triggered_txns > 0")
    .sort_values(["integration_point", "fraud_lift", "fraud_capture_rate"], ascending=[True, False, False])
    .groupby("integration_point")
    .head(5)
    .reset_index(drop=True)
)
top_candidate_rules.to_csv(OUTPUT_DIR / "recommended_candidate_rules_by_integration_point.csv", index=False)
top_candidate_rules
```

Visualization - top candidate rules

```
for integration in top_candidate_rules["integration_point"].unique():
    temp = top_candidate_rules[top_candidate_rules["integration_point"] == integration]
    temp = temp.sort_values("fraud_lift", ascending=True)

    plt.figure(figsize=(10, 6))
    plt.barh(temp["rule_name"], temp["fraud_lift"])
    plt.axvline(1, linestyle="--")
    plt.title(f"Top Candidate Rules by Fraud Lift - {integration}")
    plt.xlabel("Fraud Lift")
    plt.ylabel("Rule Name")
    save_plot(f"06_top_candidate_rules_{integration}.png")
```

15. Deduplicated cumulative rule impact

- Because one transaction can trigger multiple rules, cumulative rule impact must be deduplicated.
- Do not sum triggered transactions across rules directly.
- The portfolio view shows how fraud capture and friction change as rules are layered together.

Code - deduplicated cumulative impact

```
# Create a rule ordering. You may replace this with an ordering based on final business priorities.
rule_order = rule_backtest[["integration_point", "rule_name", "rule_priority"]].drop_duplicates()
rule_order = rule_order.sort_values(["integration_point", "rule_priority"])

cumulative_results = []

for integration in rule_order["integration_point"].unique():
    ordered_rules = rule_order[rule_order["integration_point"] == integration].sort_values("rule_priority")
    selected_rules = []

    for _, r in ordered_rules.iterrows():
        selected_rules.append(r["rule_name"])
        rule_list_sql = ", ".join(["'" + x.replace("'", "'') + "'" for x in selected_rules])
        integration_sql = str(integration).replace("'", "'')

        query = f"""
        WITH universe AS (
            SELECT *
            FROM april_velocity
            WHERE integration_point = '{integration_sql}'
        ),
        triggered AS (
            SELECT DISTINCT h.uuid
            FROM april_rule_hits h
            WHERE h.integration_point = '{integration_sql}'
            AND h.rule_name IN ({rule_list_sql})
        ),
        joined AS (
            SELECT u.*
            FROM universe u
            JOIN triggered t
            ON u.uuid = t.uuid
        )
        SELECT
            '{integration_sql}' AS integration_point,
            {len(selected_rules)} AS scenario_number,
            '{' + '.join(selected_rules).replace("'", "'') + "'" AS rules_included,
            COUNT(DISTINCT uuid) AS dedup_triggered_txns,
            SUM(fraud_direct) AS dedup_triggered_frauds,
            AVG(fraud_direct) AS precision_fraud_rate,
            COUNT(DISTINCT uuid) * 1.0 / NULLIF((SELECT COUNT(DISTINCT uuid) FROM universe), 0) AS triggered_txn_share,
            SUM(fraud_direct) * 1.0 / NULLIF((SELECT SUM(fraud_direct) FROM universe), 0) AS fraud_capture_rate,
            SUM(CASE WHEN fraud_direct = 0 THEN 1 ELSE 0 END) * 1.0 / NULLIF((SELECT COUNT(DISTINCT uuid) FROM universe), 0)
        AS friction_rate,
            COUNT(DISTINCT device_id) AS unique_devices,
            COUNT(DISTINCT account_code) AS unique_accounts
        FROM joined;
        """
        cumulative_results.append(con.execute(query).df())

cumulative_rule_impact = pd.concat(cumulative_results, ignore_index=True)
cumulative_rule_impact.to_csv(OUTPUT_DIR / "deduplicated_cumulative_rule_impact.csv", index=False)
cumulative_rule_impact.head(30)
```

Visualization - cumulative fraud capture and friction

```

for integration in cumulative_rule_impact["integration_point"].unique():
    temp = cumulative_rule_impact[cumulative_rule_impact["integration_point"] == integration]

    plt.figure(figsize=(10, 6))
    plt.plot(temp["scenario_number"], temp["fraud_capture_rate"] * 100, marker="o", label="Fraud Capture %")
    plt.plot(temp["scenario_number"], temp["friction_rate"] * 100, marker="o", label="Friction %")
    plt.title(f"Deduplicated Cumulative Rule Impact - {integration}")
    plt.xlabel("Number of Rules Included")
    plt.ylabel("Rate (%)")
    plt.legend()
    save_plot(f"07_cumulative_rule_impact_{integration}.png")

```

16. Visualizations for the updated deck

Slide/Output	Recommended visual	Message
Executive Summary	Rule portfolio KPIs	Layered rules improve precision while controlling friction.
Context & Problem	Score distribution + fraud rate	Score-only decisioning creates overblocking risk.
IPSS Distribution	Score 1-5 table and chart with Score 3 called out	Score 3 should not be ignored.
Score Capture	Fraud capture for Scores 3, 4, and 5	Scores 3 and 4 contribute to fraud capture and should be evaluated.
Reason Codes	Reason family lift bar chart	Reason families are key precision layer.
Residential Proxy	RP tier fraud lift by integration point	RP tiers behave differently and should be layered with score.
Velocity	Top velocity rules by integration point	Velocity detects automation and abuse by flow.
Candidate Rules	Exact rule names and KPIs by integration point	Rules are actionable and measurable.
Rule Portfolio	Deduplicated cumulative impact chart	Shows total impact without double-counting transactions.

17. Recommendation extension: transaction volume by rule

- Add triggered_txns and triggered_txn_share to every rule table.
- Triggered transactions show how much operational volume each rule creates.
- A high-lift rule with very low triggered volume may be useful as a surgical deny candidate, while a high-capture rule with high volume may be better suited for review or monitoring.
- Use this phrasing: This rule triggers X transactions, impacts Y% of traffic, captures Z% of fraud, and has N times the baseline risk.

18. Recommendation extension: deduplication and portfolio impact

- Evaluate individual rules and the cumulative portfolio separately.
- Deduplicate cumulative counts at the transaction level using UUID.
- Show fraud capture and friction as rules are added sequentially.
- This answers the client question: If we implement the recommended rule package, how much fraud do we capture and how much customer friction do we create?

19. Final client-facing recommendation table

- The final table should be grouped by integration point.
- Include exact rule name, recommended action, triggered transactions, triggered transaction share, fraud rate/precision, fraud lift, fraud capture, friction, unique devices, and unique accounts.
- Sort rules by business priority, not only by fraud lift. A practical order is: Deny candidates, Review/Step-up candidates, Monitor/secondary signals, Not recommended.

Integration Point	Rule Name	Action	Triggered Txns	Txn Share	Fraud Rate	Fraud Lift	Fraud Capture	Friction
vb_dda_login	Score 3/4 + Anonymous Privacy	Review / Step-up	X	Y%	Z%	Nx	A%	B%
vb_dda_application	Score 4 + RP3	Deny candidate if	X	Y%	Z%	Nx	A%	B%

		friction tolerance allows						
vb_dda_v2a_receiver	Score 3/4 + High Device Velocity 24h	Review	X	Y%	Z%	Nx	A%	B%

20. Suggested client storyline

- IPScore is effective as a risk segmentation signal, but it should not be used as a standalone decisioning rule.
- April 2026 results show that Score 3 and Score 4 deserve deeper review, especially when combined with Anonymous / Privacy, Attack / Bot / Spam, Residential Proxy, or high velocity behavior.
- Reason families are the strongest precision layer, while velocity captures automation and repeated behavior.
- The recommended strategy is integration-point-specific layered decisioning using Score + Reason Family + Velocity.
- Each recommended rule should be measured by triggered transactions, fraud capture, fraud lift, and customer friction.
- Cumulative rule impact must be deduplicated to avoid overstating the operational impact.

Final message: Client-facing bottom line: Avoid blanket score-only deny rules. Implement integration-point-specific layered rules and monitor each rule by triggered volume, fraud capture, lift, and friction.

21. Full Python/DuckDB code block

How to use: The earlier sections break the analysis into logical modules. The block below lists the expected output files and the order in which the code should be run.

Execution checklist

```
# Recommended execution order
# 1) Setup and create april_base
# 2) Generate baseline_april2026.csv
# 3) Generate score_summary_april2026.csv and score visuals
# 4) Generate supplemental_reason_family_groupings.csv
# 5) Generate reason_family_performance_april.csv and visuals
# 6) Generate april_enriched with reason flags
# 7) Generate residential_proxy_rp_tier_summary_april.csv
# 8) Generate april_velocity
# 9) Generate velocity_thresholds_by_integration_point.csv
# 10) Generate velocity_rule_backtest_by_integration_point.csv
# 11) Generate recommended_velocity_rules_by_integration_point.csv
# 12) Generate april_rule_hits
# 13) Generate candidate_rule_backtest_by_integration_point.csv
# 14) Generate recommended_candidate_rules_by_integration_point.csv
# 15) Generate deduplicated_cumulative_rule_impact.csv
# 16) Use visualizations from OUTPUT_DIR in the updated presentation

# Final output files:
# - baseline_april2026.csv
# - score_summary_april2026.csv
# - score_3_4_comparison.csv
# - supplemental_reason_family_groupings.csv
# - reason_family_performance_april.csv
# - residential_proxy_rp_tier_summary_april.csv
# - velocity_thresholds_by_integration_point.csv
# - velocity_rule_backtest_by_integration_point.csv
# - recommended_velocity_rules_by_integration_point.csv
# - candidate_rule_backtest_by_integration_point.csv
# - recommended_candidate_rules_by_integration_point.csv
# - deduplicated_cumulative_rule_impact.csv
# - PNG charts saved under client_facing_april2026_outputs/
```